Double-click (or enter) to edit

```
import tensorflow as tf


num_gpus_available = len(tf.config.experimental.list_physical_devices('GPU'))
print("Num GPUs Available: ", num_gpus_available)
# assert num_gpus er_available > 0
```

```
!pip install transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.11/dist-packages (4.47.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from transformers) (3.17.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.24.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.27.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.21.0)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.5.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.11/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0,>=0.24.0->trans
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0,>=0.2
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (3
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2024.12
```

```
import pandas as pd
import numpy as np


import nltk
import re
nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
df = pd.read_csv('/content/amazon_reviews_us_Major_Appliances_v1_00.tsv', sep='\t', quoting=3)


df.head()
```

	marketplace	customer_id	review_id	product_id	product_parent	product_title	product_category	star_rating	helpful
0	US	16199106	R203HPW78Z7N4K	B0067WNSZY	633038551	FGGF3032MW Gallery Series 30" Wide Freestandin...	Major Appliances	5	
1	US	16374060	R2EAIGVLEALSP3	B002QSXK60	811766671	Best Hand Clothes Wringer	Major Appliances	5	
2	US	15322085	R1K1CD73HHLILA	B00EC452R6	345562728	Supco SET184 Thermal Cutoff Kit	Major Appliances	5	
3	US	32004835	R2KZBMOFRMYOPO	B00MVVIF2G	563052763	Midea WHS-160RB1 Compact Single Reversible Doo...	Major Appliances	5	
4	US	25414497	R6BIZOZY6UD01	B00IY7BNUW	874236579	Avalon Bay Portable Ice Maker	Major Appliances	5	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df.shape
```

```
(96901, 15)
```

```
df["Sentiment"] = df["star_rating"].apply(lambda score: "positive" if score >= 3 else "negative")
df['Sentiment'] = df['Sentiment'].map({'positive':1, 'negative':0})
```

```
df = df[["review_body", "Sentiment"]]
```

```
df.shape
```

```
(96901, 2)
```

```
n = 54975
df.drop(df.tail(n).index,
        inplace = True)
```

```
index = df.index
number_of_rows = len(index)
print(number_of_rows)
```

```
41926
```

```
# Count the number of each sentiment
sentiment_counts = df['Sentiment'].value_counts()

# Get the counts of positive and negative sentiments
positive_count = sentiment_counts.get(1, 0)  # 1 represents positive
negative_count = sentiment_counts.get(0, 0)  # 0 represents negative

print(f"Positive Sentiment Count: {positive_count}")
print(f"Negative Sentiment Count: {negative_count}")
```

```
Positive Sentiment Count: 32778
Negative Sentiment Count: 9148
```

```
reviews = df['review_body'].values.tolist()
labels = df['Sentiment'].tolist()
```

```
from sklearn.model_selection import train_test_split
training_sentences, validation_sentences, training_labels, validation_labels = train_test_split(reviews, labels, test_size=.2)
```

```
len(validation_sentences)
```

```
8386
```

validation_sentences

```
['Im just glad to have water. Hot water is super. Indeed. No complains. It is short for myself and partner at home but no hassle.',
 'product fit perfect and works like a charm.  I am very pleased.',
 "I dont know which model I have, but my front loading washer dryer is the worst pile of crap I've  ever bought. The fact that they could sell something sooo bad means it is a failing mismanaged company. The washer gets moldy all around the rubber gasket, and UNDER the rubber gasket. All the bleach and q-tips in the world won't remove it. Then they send me letter after few years wanting to reimburse me in case I have had a dryer fire. Half the time when you try to remove the filter on the dryer the lint actually falls off the filter and falls into the filter opening and down past a rubber flange. I suppose we've  been lucky because I clean that opening out regularly with a bent clothes hanger! Mind boggling!!<br /><br />Any company that is so poorly run to release such grossly inadequate, inferior, malfunctioning products, with no concern for their customers or future sales is a company I will never ever buy another product from. They could not have possibly tested this more than once or twice to be able to pronounce this usable. Oh and I had the misfortune to purchase a stove made by them around the same time. The burners no longer light by themselves.",
 "Does the job as expected and prevents strains on my stainless steel appliances. Only issue is these covers don't fully provide coverage (just a little too short on diameter)  to my fridge handles exposing the welcro a little bit.",
 "My GE fridge decided to stop making Ice a while ago.  I ordered one of these from another vendor at that time and they shipped a part for an Electroloux fridge.  So I sent that back and gave up on the project for a while.  I was just buying bags of ice from the grocery store and filling up my bin.  I finally decided to dig into things once again and ordered this part.<br /><br />The only thing that is different on this device over my old one is the &#34;On/Off&#34; switch.  Other than that it's exactly the same.  The old one came off easily enough.  I used a nut bit on my drill to remove this because my ratchet set didn't have anything small enough.  I did use a part off of the old one, the part was a plastic piece that channeled the water from the spout to where the cubes are made.  That un-clipped easily and fit right on the new one.  I got the new one installed with the existing screws and the power plugged in as it needed to.<br /><br />Once everything was installed I turned it on and it started making ice right away.  At this point the bin is full of ice and I no longer have to buy bags and fill the thing up.  I honestly don't know why I waited this long.  Works great in my GE fridge.",
 'Great product for a great price.<br /><br />These fit our water dispenser on our fridge and they last for about four months. Being able to purchase a three pack once a year just makes things easier for me.<br /><br />Thumbs up on these for price and quality.',
 'Great product! High quality and powerful!',
 'Greatest portable dishwasher ever!  We have had it for 2 weeks.  We owned the counter top kind before and this is much better. Washes the dishes great. The no dry cycle does not bother us at all.  We just let the dishes sit until they dry and there are no spots. The only complaint I have is that the connecter hose is too short to reach our sink.  We are going to visit Home Depot to figure out how to extend it.',
 'came as promised. Helpful staff. Fantastic machines!',
 'Beeper is so annoying, it beeps 3 long beeps and I have called there is no way to alter it. The most people usually set a microwave for is 1-2 minutes so who needs this long loud beep, can you forget that quickly you put something in the microwave esp. since you are still standing right in front of it? Also the door handle has broken twice in under 18 months, first time it was covered, then when I called when it rebroke I was told it was no longer under warranty even though the part I was calling about was only 2 months installed. The part should have its own warranty. Now I am extra gentle with the door but I use it less for both these reasons.',
 'This is the best purchase I have made in a long time. It does a better job than most full-sized dishwashers. Of course, I allow my boxer to &#34;pre-lick&#34; each plate prior to loading it! Then he is lulled to sleep by the gentle rhythmic sound of it washing &#34;his dishes&#34;. After two months, no problems. Both I and my dog love it!',
 'the video on how to install and this switch work great it took less than 5 minutes to take a part and put it together...',
 "There are no instructions with this product.  And no instructions on the dryer documentation.  Yes, this product worked perfectly as a replacement.  I had been aware that the heating element on my 2002 Whirlpool dryer was probably the culprit based on diminishing performance.<br /><br /> When it stopped heating, I went online and watched a very entertaining &#34;DIY&#34; video.  Believing that I could replace this myself, I ordered the part, and when it arrived, I set about installing it, per the video. But look closely at the picture of the part-- it has a &#34;hole&#34; on the upper right hand side.  My installed  dryer part has a black insert that has two contact points, that have a metal connection to the positive outlet of the element (it's a jumper element).<br /><br /> That is when I discovered that my part was &#34;not quite the same&#34; (had the extra piece&#34; that the replacement didn't have.  After inspecting it closely, I (as it turns out, wisely) decided I better not chance it, and decided to call a repairman.  I was frustrated that I could not do this myself, and asked the repairman  if I should have been able to.<br /><br /> His response was revealing:  &#34;A lot of customers try to do this, and they blow out the entire thermocouple on the dryer--you could have done that---the 220v will blow out the new heating element as well (ruin the dryer electronics, basically).  Then customers try and claim it was a defective part, but we know what really happened, and we don't give them the credit for the heating element.&#34;  When I asked if I should have been able to do this as a DIY project, he told me, &#34;Honestly, no&#34;, you needed something for this (the jumper piece)  that you don't have, but I have on my truck.  I
```

validation_labels

```
        1,
        1,
        1,
        1,
        0,
        1,
        0,
        1,
        1,
        1,
        1,
        1,
        0,
        1,
        1,
        1,
        0,
        0,
        1,
        1,
        1,
        0,
        1,
        1,
        1,
        0,
        1,
        0,
        0,
        1,
        1,
        0,
        1,
        1,
        1,
```

```
from transformers import TFDistilBertModel, DistilBertTokenizer
```

```
from transformers import TFDistilBertModel, DistilBertTokenizer
import tensorflow as tf

# Load the DistilBERT tokenizer and model
model_name = "distilbert-base-uncased"
tokenizer = DistilBertTokenizer.from_pretrained(model_name)
encoder = TFDistilBertModel.from_pretrained(model_name)

# Freeze the layers of DistilBERT
for layer in encoder.layers:
    layer.trainable = False

# Input layers for input_ids and attention_mask
input_ids = tf.keras.layers.Input(shape=(128,), dtype=tf.int32, name='input_ids')  # Explicitly named input layer
attention_mask = tf.keras.layers.Input(shape=(128,), dtype=tf.int32, name='attention_mask')  # Explicitly named input layer

# Forward pass through the encoder (DistilBERT)
outputs = tf.keras.layers.Lambda(
    lambda x: encoder(input_ids=x[0], attention_mask=x[1]).last_hidden_state,  # Access last_hidden_state directly
    output_shape=(128, 768)  # Specify the output shape: (sequence_length, hidden_size)
)([input_ids, attention_mask])

# Global Average Pooling to reduce the sequence length and retain embeddings
pooled_output = tf.keras.layers.GlobalAveragePooling1D()(outputs)

# Adding two Dense layers without regularization
dense_layer_1 = tf.keras.layers.Dense(
    256,
    activation='relu'
)(pooled_output)

dense_layer_2 = tf.keras.layers.Dense(
    128,
    activation='relu'
)(dense_layer_1)

# Output layer for binary classification
output_layer = tf.keras.layers.Dense(1, activation='sigmoid')(dense_layer_2)

# Build the model
model = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=output_layer)

# Display model summary
model.summary()
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 3.77kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 5.15MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 3.01MB/s]
config.json: 100%  483/483 [00:00<00:00, 32.3kB/s]
model.safetensors: 100%  268M/268M [00:01<00:00, 207MB/s]
```

```
Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFDistilBertModel: ['vocab_layer_norm.weight', 'v
- This IS expected if you are initializing TFDistilBertModel from a PyTorch model trained on another task or with another architectu
- This IS NOT expected if you are initializing TFDistilBertModel from a PyTorch model that you expect to be exactly identical (e.g.
All the weights of TFDistilBertModel were initialized from the PyTorch model.
If your task is similar to the task the model of the checkpoint was trained on, you can already use TFDistilBertModel for predictior
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_ids (InputLayer)	(None, 128)	0	-
attention_mask (InputLayer)	(None, 128)	0	-
lambda (Lambda)	(None, 128, 768)	0	input_ids[0][0], attention_mask[0][0]
global_average_pooling1d (GlobalAveragePooling1D)	(None, 768)	0	lambda[0][0]
dense (Dense)	(None, 256)	196,864	global_average_poolin…
dense_1 (Dense)	(None, 128)	32,896	dense[0][0]
dense_2 (Dense)	(None, 1)	129	dense_1[0][0]

Total params: 229,889 (898.00 KB)
Trainable params: 229,889 (898.00 KB)
Non-trainable params: 0 (0.00 B)

```
model_name = "distilbert-base-uncased"
tokenizer = DistilBertTokenizer.from_pretrained(model_name)

# Example training and validation sentences (replace these with your actual data)

training_sentences = [str(sentence) for sentence in training_sentences]
validation_sentences = [str(sentence) for sentence in validation_sentences]
# Tokenize the sentences, ensure padding='max_length' and truncation=True
train_encodings = tokenizer(training_sentences, padding='max_length', truncation=True, max_length=128, return_tensors="tf")
val_encodings = tokenizer(validation_sentences, padding='max_length', truncation=True, max_length=128, return_tensors="tf")

# Convert to TensorFlow Dataset
train_dataset = tf.data.Dataset.from_tensor_slices((
    {
        'input_ids': train_encodings['input_ids'],
        'attention_mask': train_encodings['attention_mask']
    },
    training_labels
))

val_dataset = tf.data.Dataset.from_tensor_slices((
    {
        'input_ids': val_encodings['input_ids'],
        'attention_mask': val_encodings['attention_mask']
    },
    validation_labels
))

# Shuffle and batch the datasets
train_dataset = train_dataset.shuffle(100).batch(128)
val_dataset = val_dataset.batch(128)


from tensorflow.keras.optimizers import Adam
import tensorflow as tf

# Compile the model with a lower learning rate
optimizer = Adam(learning_rate=1e-5)
model.compile(optimizer=optimizer, loss=tf.keras.losses.BinaryCrossentropy(), metrics=['accuracy', tf.keras.metrics.Precision(), tf.kera
```

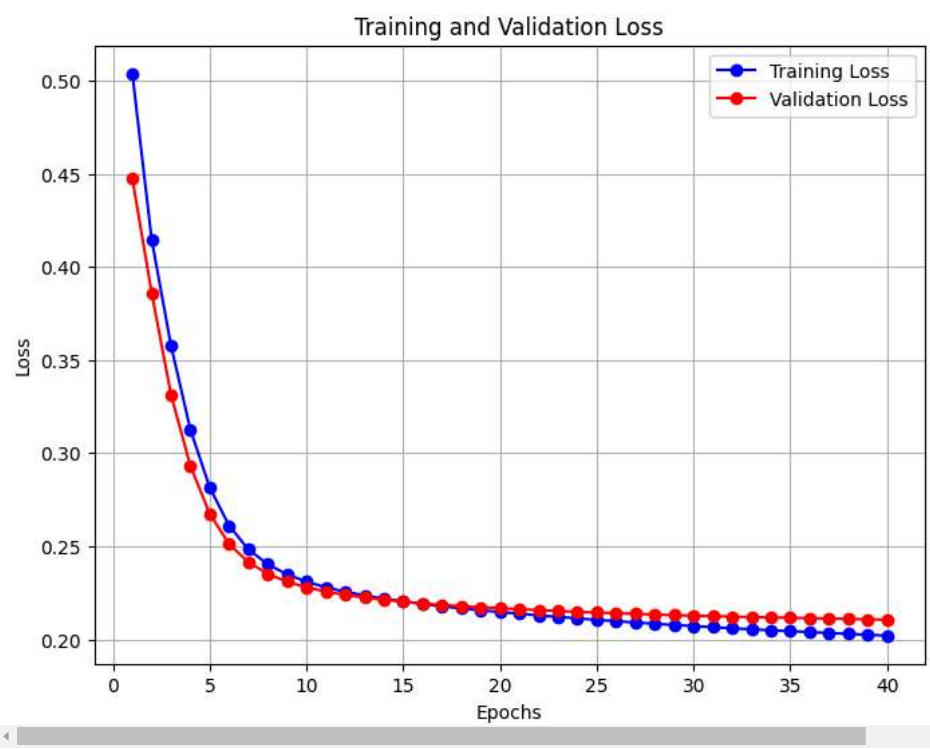
```
# Train the model
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=40
)
```

```
Epoch 1/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 164s 554ms/step - accuracy: 0.7825 - loss: 0.5446 - precision: 0.7826 - recall: 0.9998 - val_accurac
Epoch 2/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 192s 546ms/step - accuracy: 0.7870 - loss: 0.4300 - precision: 0.7864 - recall: 0.9994 - val_accurac
Epoch 3/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 543ms/step - accuracy: 0.8350 - loss: 0.3721 - precision: 0.8342 - recall: 0.9851 - val_accurac
Epoch 4/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 155s 592ms/step - accuracy: 0.8610 - loss: 0.3237 - precision: 0.8641 - recall: 0.9761 - val_accurac
Epoch 5/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 144s 547ms/step - accuracy: 0.8779 - loss: 0.2901 - precision: 0.8871 - recall: 0.9672 - val_accurac
Epoch 6/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.8874 - loss: 0.2679 - precision: 0.9015 - recall: 0.9611 - val_accurac
Epoch 7/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 543ms/step - accuracy: 0.8921 - loss: 0.2533 - precision: 0.9106 - recall: 0.9561 - val_accurac
Epoch 8/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.8948 - loss: 0.2449 - precision: 0.9158 - recall: 0.9532 - val_accurac
Epoch 9/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 543ms/step - accuracy: 0.8964 - loss: 0.2389 - precision: 0.9183 - recall: 0.9523 - val_accurac
Epoch 10/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.8982 - loss: 0.2348 - precision: 0.9206 - recall: 0.9521 - val_accurac
Epoch 11/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 215s 594ms/step - accuracy: 0.8995 - loss: 0.2315 - precision: 0.9228 - recall: 0.9512 - val_accurac
Epoch 12/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 144s 546ms/step - accuracy: 0.9007 - loss: 0.2291 - precision: 0.9241 - recall: 0.9512 - val_accurac
Epoch 13/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 155s 591ms/step - accuracy: 0.9014 - loss: 0.2270 - precision: 0.9250 - recall: 0.9512 - val_accurac
Epoch 14/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 144s 546ms/step - accuracy: 0.9030 - loss: 0.2249 - precision: 0.9280 - recall: 0.9498 - val_accurac
Epoch 15/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.9037 - loss: 0.2237 - precision: 0.9276 - recall: 0.9513 - val_accurac
Epoch 16/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.9038 - loss: 0.2222 - precision: 0.9279 - recall: 0.9510 - val_accurac
Epoch 17/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 545ms/step - accuracy: 0.9040 - loss: 0.2211 - precision: 0.9286 - recall: 0.9504 - val_accurac
Epoch 18/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 543ms/step - accuracy: 0.9049 - loss: 0.2197 - precision: 0.9302 - recall: 0.9498 - val_accurac
Epoch 19/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 543ms/step - accuracy: 0.9060 - loss: 0.2187 - precision: 0.9301 - recall: 0.9515 - val_accurac
Epoch 20/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 203s 547ms/step - accuracy: 0.9065 - loss: 0.2177 - precision: 0.9302 - recall: 0.9520 - val_accurac
Epoch 21/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.9059 - loss: 0.2171 - precision: 0.9289 - recall: 0.9527 - val_accurac
Epoch 22/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 544ms/step - accuracy: 0.9082 - loss: 0.2159 - precision: 0.9323 - recall: 0.9518 - val_accurac
Epoch 23/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 545ms/step - accuracy: 0.9078 - loss: 0.2152 - precision: 0.9310 - recall: 0.9529 - val_accurac
Epoch 24/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 543ms/step - accuracy: 0.9089 - loss: 0.2143 - precision: 0.9323 - recall: 0.9528 - val_accurac
Epoch 25/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 545ms/step - accuracy: 0.9095 - loss: 0.2136 - precision: 0.9340 - recall: 0.9516 - val_accurac
Epoch 26/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 203s 548ms/step - accuracy: 0.9097 - loss: 0.2128 - precision: 0.9327 - recall: 0.9534 - val_accurac
Epoch 27/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 143s 545ms/step - accuracy: 0.9110 - loss: 0.2118 - precision: 0.9341 - recall: 0.9536 - val_accurac
Epoch 28/40
263/263 ━━━━━━━━━━━━━━━━━━━━ 203s 547ms/step - accuracy: 0.9109 - loss: 0.2113 - precision: 0.9335 - recall: 0.9542 - val_accurac
Epoch 29/40
```

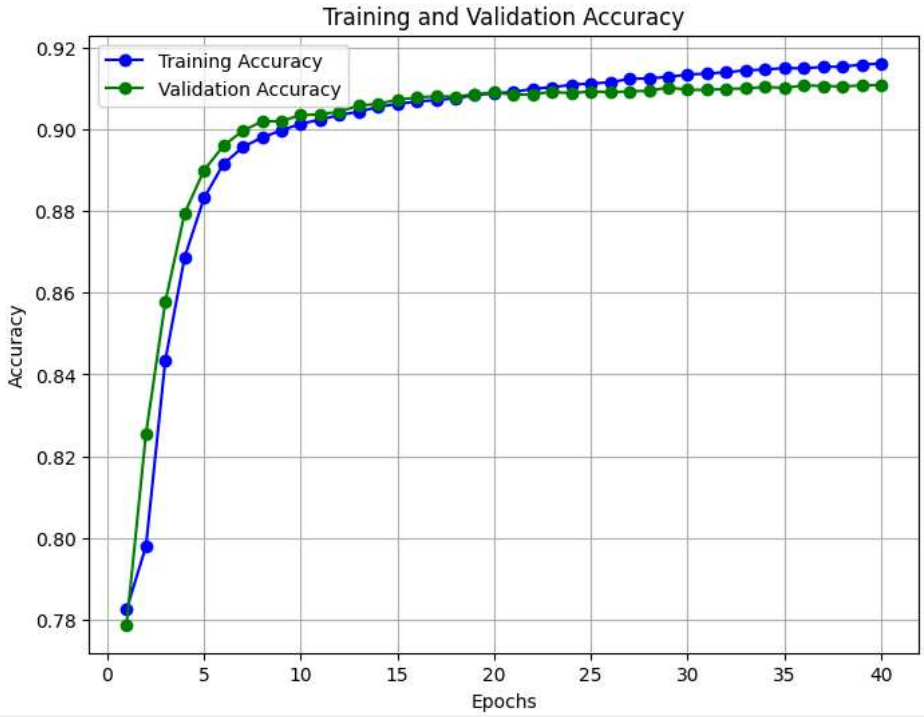
```
test_sentence = input("Enter a sentence for sentiment analysis: ")

# Tokenize and predict
predict_input = tokenizer(test_sentence, truncation=True, padding='max_length', max_length=128, return_tensors="tf")
probability = model.predict({'input_ids': predict_input['input_ids'], 'attention_mask': predict_input['attention_mask']})[0][0]  # Get th

# Determine the label based on the probability
label = 1 if probability > 0.5 else 0  # Positive if probability > 0.5, otherwise Negative

# Define labels
labels = ['Negative', 'Positive']

# Print sentiment result
print(f"Sentiment: {labels[label]} (Confidence: {probability:.2f})")
print(f"Sentiment: {labels[label]}")
```

```
Enter a sentence for sentiment analysis: Excellent battery life – lasts almost 3 days on a single charge
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
Sentiment: Positive (Confidence: 0.94)
Sentiment: Positive
```

```
import matplotlib.pyplot as plt

# Extract the loss values from the history object
training_loss = history.history['loss']
validation_loss = history.history['val_loss']

# Create a range for the epochs
epochs = range(1, len(training_loss) + 1)

# Plot the training and validation loss
plt.figure(figsize=(8, 6))
plt.plot(epochs, training_loss, label='Training Loss', color='blue', marker='o')
plt.plot(epochs, validation_loss, label='Validation Loss', color='red', marker='o')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```

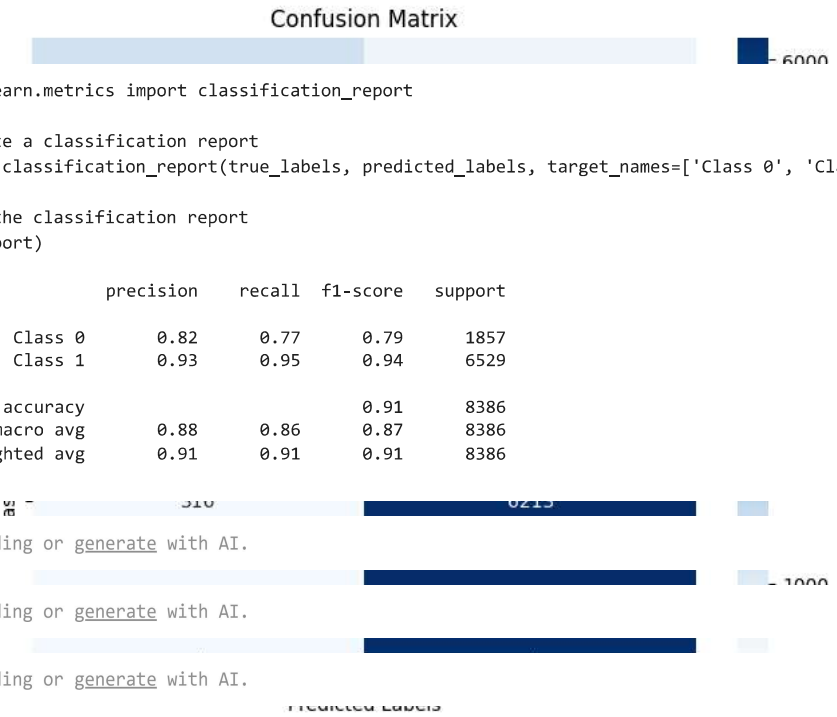



```
import matplotlib.pyplot as plt

# Extract the accuracy values from the history object
training_accuracy = history.history['accuracy']
validation_accuracy = history.history['val_accuracy']

# Create a range for the epochs
epochs = range(1, len(training_accuracy) + 1)

# Plot the training and validation accuracy
plt.figure(figsize=(8, 6))
plt.plot(epochs, training_accuracy, label='Training Accuracy', color='blue', marker='o')
plt.plot(epochs, validation_accuracy, label='Validation Accuracy', color='green', marker='o')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```




```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Get predictions from the model
predictions = model.predict(val_dataset)  # Use validation or test dataset

# Step 2: Convert predictions to binary class labels
predicted_labels = (predictions > 0.5).astype(int).flatten()  # Binary classification

# Step 3: Extract true labels from the validation dataset
true_labels = np.concatenate([y for x, y in val_dataset], axis=0)  # Adjust for your dataset format

# Step 4: Generate the confusion matrix
conf_matrix = tf.math.confusion_matrix(labels=true_labels, predictions=predicted_labels).numpy()

# Step 5: Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=['Class 0', 'Class 1'], yticklabels=['Class 0', 'Class 1'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.show()
```

```
66/66 ━━━━━━━━━━━━━━━━━━━━ 34s 484ms/step
```




```
from sklearn.metrics import classification_report

# Generate a classification report
report = classification_report(true_labels, predicted_labels, target_names=['Class 0', 'Class 1'])

# Print the classification report
print(report)
```

```
              precision    recall  f1-score   support

     Class 0       0.82      0.77      0.79      1857
     Class 1       0.93      0.95      0.94      6529

    accuracy                           0.91      8386
   macro avg       0.88      0.86      0.87      8386
weighted avg       0.91      0.91      0.91      8386
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.